

SLIC Segmentation Algorithm: Performance Optimization through Concurrent Programming with OpenMP

Marco De Stefano

April 2026



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Table of Contents

1 Introduction

► Introduction

► Approach

► Experiments

► Conclusions

Introduction to SLIC Segmentation

1 Introduction

- **Simple Linear Iterative Clustering (SLIC) Segmentation:** Computer Vision Algorithm.
- **Goal:** Groups pixels into superpixels similar in color and proximity, preserving boundaries.
- **Problem:** Sequential execution is a bottleneck for high-resolution images.
- **Complexity:** $O(N)$.
- **Challenge:** Accelerate the algorithm using CPU Multithreading with OpenMP.

SLIC Segmentation Example

1 Introduction



Figure: Original Image



Figure: Segmented Image

Complexity of the Algorithm

1 Introduction

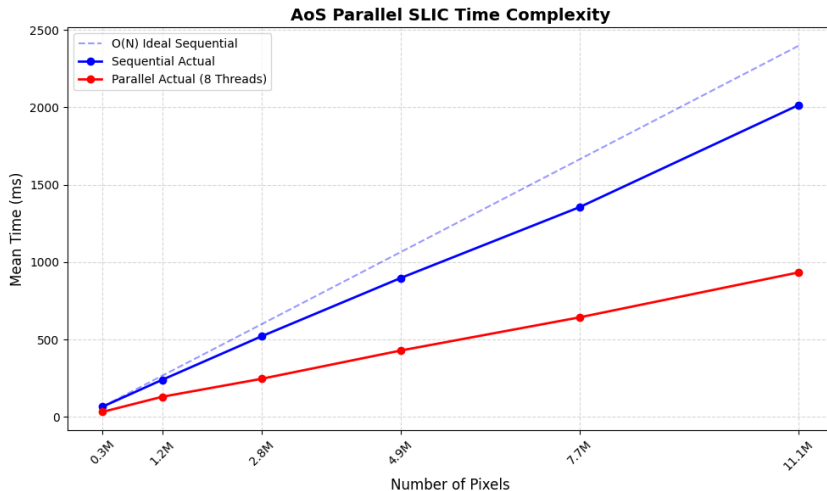


Table of Contents

2 Approach

► Introduction

► **Approach**

► Experiments

► Conclusions

Experiment Setup & Baseline

2 Approach

- **Hardware Environment:**
 - **CPU:** Apple Silicon ARM (8 Cores).
 - **Topology:** Asymmetric (4 P-Cores + 4 E-Cores).
 - **Cache:** 128-byte Cache Line size on P-Cores.
 - 8 GB Unified Memory
- **Software & Tools:**
 - **Language:** C++ & OpenMP.
 - **Compiler Flags:** `-O3, -ffast-math`.
 - **Analysis & Plotting:** Python, Matplotlib.
- **Dataset:**
 - Berkeley Segmentation Dataset (640×480).
 - Target Superpixels (K): 1000.

Target Phases to Parallelize

2 Approach

- **Initialization (Seq):** Grid setup and gradient perturbation.
- **Assignment (Parallel):** Find the nearest cluster center for each pixel.
- **Update Centroids (Parallel):**
 - Recalculate cluster centers.
 - High risk of *Race Conditions*.
- **Enforce Connectivity (Seq):**
 - Merges disjoint regions (BFS approach).
 - Kept sequential to avoid massive synchronization overhead.

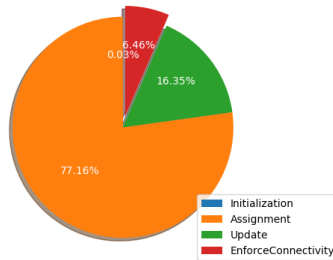


Figure: Parallelizable vs Sequential Workload

Architectural Tuning & Optimization

2 Approach

- **NUMA-Aware Initialization (First-Touch):**
 - Parallel array initialization physically maps memory to the local core cluster, reducing latency.
- **Data Layout & Vectorization:**
 - **SoA:** Enables -O3 auto-vectorization for color computations.
 - **AoS:** Explicit `alignas(128)` padding eliminates False Sharing.
 - **Explicit SIMD:** `#pragma omp simd` accelerates centroid averaging.
- **2D Cache Tiling ($2S \times 2S$):**
 - Pins local centroids in L1/L2 caches to maximize **Temporal Locality**.

Assignment Method

2 Approach

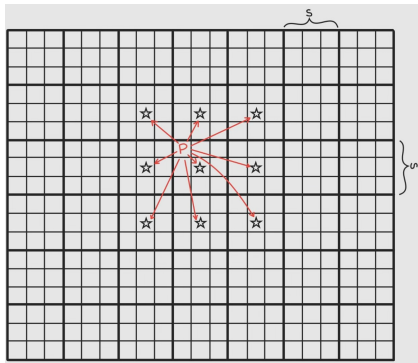


Figure: Pixel Centric (Chosen)

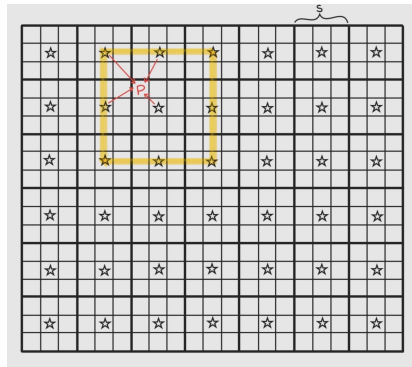


Figure: Centroid Centric (Discarded)

Table of Contents

3 Experiments

► Introduction

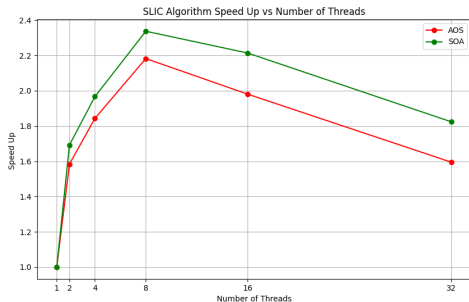
► Approach

► Experiments

► Conclusions

Thread Scalability & Oversubscription

3 Experiments



- The "Sweet Spot" (8 Threads):

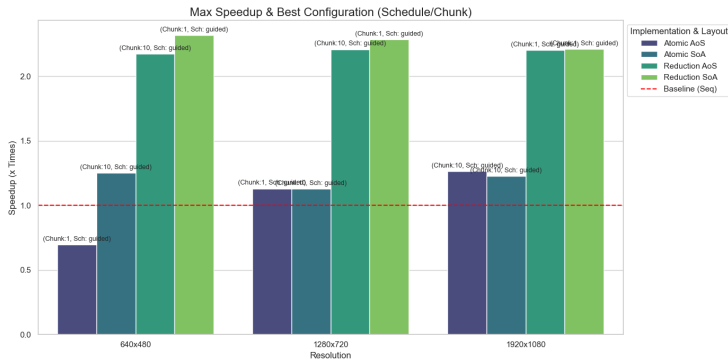
- Peak strictly matches the hardware physical cores (no Hyper-Threading).

- Hardware Oversubscription (> 8):

- **Context Switching:** OS wastes cycles swapping thread states.
- **Cache Thrashing:** Suspended threads lose their L1 data.
- **AoS Padding Penalty:** The alignas padding in AoS amplifies the memory bandwidth waste during thrashing.

Best Synchronization & Scheduling

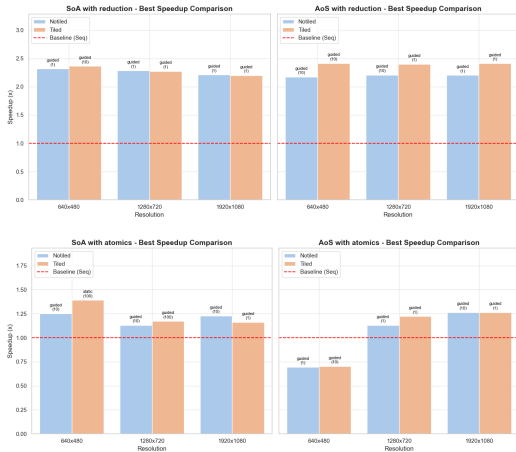
3 Experiments



- **Reductions > Atomics:** Atomics suffer from *True Sharing* on centroid accumulators, stalling the CPU pipeline. Reductions use thread-private workspaces.
- **Guided Scheduler:** Perfectly balances the asymmetric workload between P-Cores and E-Cores.

The Tiling Showdown (2D Cache Blocking)

3 Experiments



- **Goal:** Use $2S \times 2S$ Tiling in the Assignment Phase to pin the 9 local centroids in the L1/L2 cache.
- **AoS Resilience (The Winner):**
 - *Perfect Marriage:* Centroids stay pinned, while "tourist" pixels are loaded contiguously (L, a, b, x, y in one fetch).
- **SoA Collapse:**
 - *Structural Fragmentation:* Scattered across separate arrays.
 - 2D strided access breaks the hardware prefetcher.

Table of Contents

4 Conclusions

► Introduction

► Approach

► Experiments

► Conclusions

Final Conclusions

4 Conclusions

- **The Winning Configuration (2.41x Speedup):**
 - **AoS Layout + Tiling + Reductions + Guided Scheduler.**
 - Maximizes spatial/temporal locality and handles the P/E Core asymmetry without True Sharing.
- **Memory Access Dictates Layout:**
 - **SoA** is strictly superior for *Linear Scans* (relies on HW Prefetcher).
 - **AoS** is strictly superior for *2D/Strided Access* (relies on pixel-level spatial locality).
- **Amdahl's Limit:**
 - The 6% sequential time (Initialization & Enforce Connectivity) caps the maximum theoretical speedup at $\sim 5.6\times$ on 8 cores.
 - Achieving $2.41\times$ indicates the bottleneck is hardware memory bandwidth, not the sequential phases.
- **Future Works:** GPU Porting via CUDA (massive memory bandwidth) and Centroid-Centric approach exploration.

Thank you for your attention!

Any Questions?